



IIT Kharagpur



IIT Madras



IIT Goa



IIT PALAKKAD

IIT Palakkad

Applied Accelerated Artificial Intelligence



Fundamentals of Accelerating Deployments

Satyadhyan Chickerur, PhD

Professor

School of Computer Science and Engineering

KLE Technological University

NVIDIA DLI Ambassador/Instructor

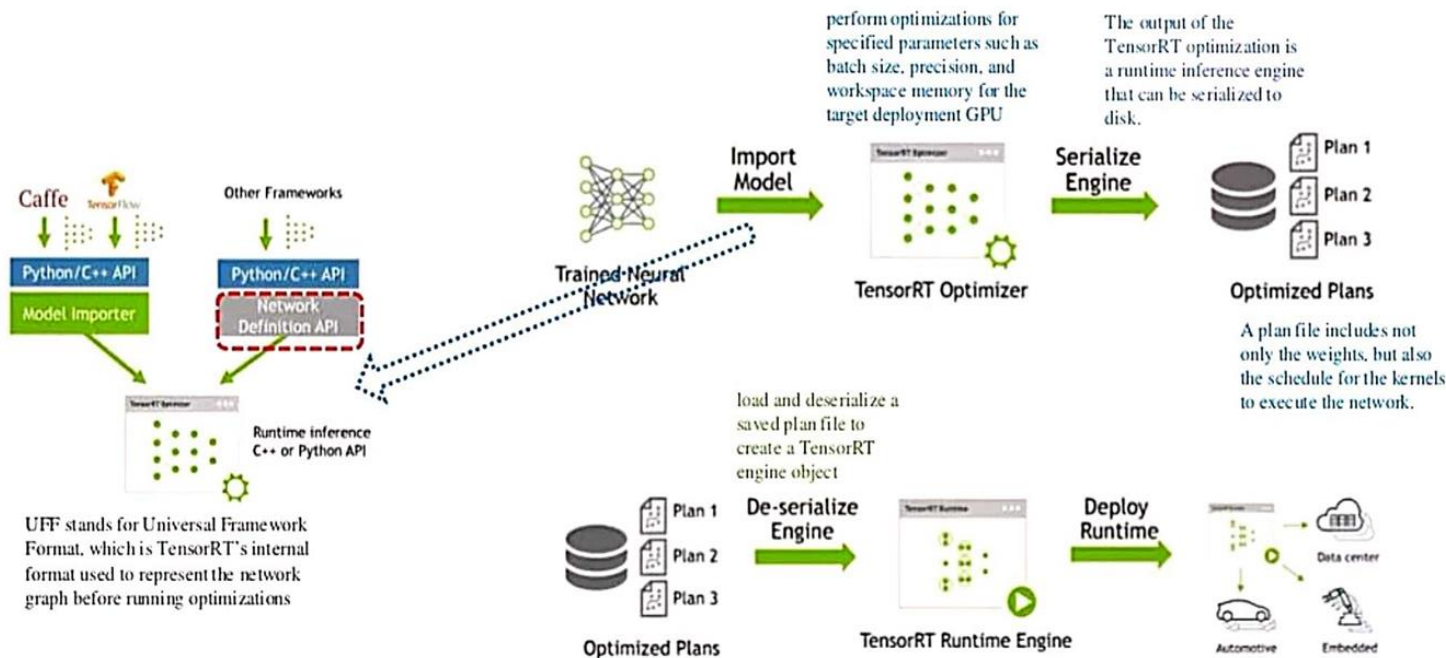


National
Supercomputing
Mission



Centre for
Development of
Advanced Computing

Deploying a model with TensorRT





TensorRT

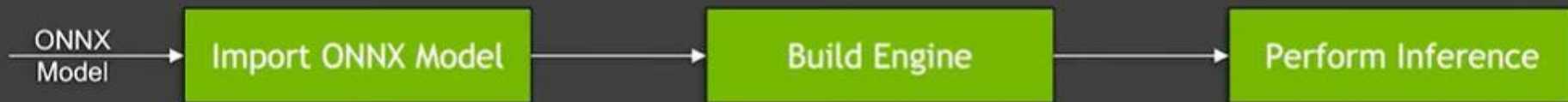
TensorRT supports the following layer types.

- Convolution: 2D
- Activation: ReLU, tanh and **sigmoid**
- **Pooling**: max and average
- ElementWise: sum, product or max of two tensors
- LRN: cross-channel only
- Fully-connected: with or without bias
- SoftMax: cross-channel only
- Deconvolution



Steps for TensorRT application.

1. Convert the pretrained image segmentation PyTorch model into ONNX.
2. Import the ONNX model into TensorRT.
3. Apply optimizations and generate an engine.
4. Perform inference on the GPU.





contd..

The application uses the following components in TensorRT:

- **ONNX parser:** Takes a converted any trained model into the ONNX format as input and populates a network object in TensorRT.
- **Builder:** Takes a network in TensorRT and generates an engine that is optimized for the target platform.
- **Engine:** Takes input data, performs inferences, and emits inference output.
- **Logger:** Associated with the builder and engine to capture errors, warnings, and other information during the build and inference phases.



Performance metrics

- TensorRT powered NVIDIA's wins across all performance tests in the industry-standard **MLPerf Inference** benchmark.
- It accelerates every model across the data center and edge in **computer vision**, speech-to-text, natural language understanding (BERT), and recommender systems.





Framework Integration.

Supports all major frameworks

- TensorRT is integrated with PyTorch and TensorFlow so you can achieve 6x faster inference with 1 line of code.





TensorFlow-TensorRT(TF-TRT)

TensorRT Integration Speeds Up TensorFlow Inference.

- TensorFlow™ integration with TensorRT™ (TF-TRT) optimizes and executes compatible subgraphs, allowing TensorFlow to execute the remaining graph.
- TensorRT will parse the model and apply optimizations to the portions of the graph wherever possible.
- TensorRT sped up TensorFlow inference by 8x for low latency runs of the ResNet-50 benchmark.
- Note: These graph optimizations do not change the underlying computation in the graph; instead, they look to restructure the graph to perform the operations much faster and more efficiently.

Fig : Workflow Diagram when using TensorRT within TensorFlow During inference.



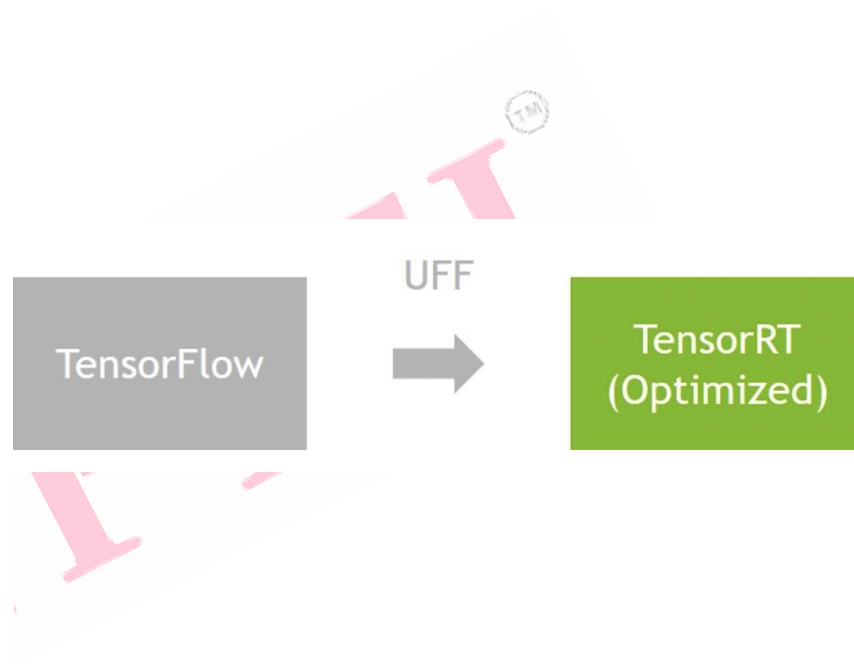
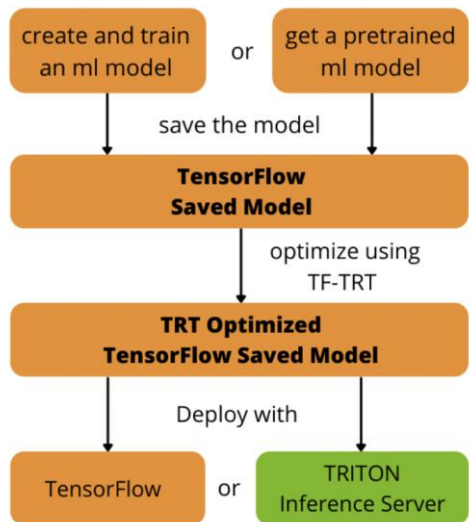
Benefits of Integrating TensorFlow with TensorRT

TensorRT optimizes the largest subgraphs possible in the TensorFlow graph. The more compute in the subgraph, the greater benefit obtained from TensorRT. You want most of the graph optimized and replaced with the fewest number of TensorRT nodes for best performance. Based on the operations in your graph, it's possible that the final graph might have more than one TensorRT node.





Workflow of TF-TRT





You will need to create a SavedModel (or frozen graph) out of a trained TensorFlow model and give that to the Python API of TF-TRT, which then :

- returns the TensorRT optimized SavedModel (or frozen graph).
- replaces each supported subgraph with a TensorRT optimized node (called TRTEngineOp), producing a new TensorFlow graph.

The following code snippets show how to use TF-TRT in TensorFlow 1.x (with default configuration) on a given model for each of the formats: SavedModel or a frozen graph

SavedModel format:

```
from tensorflow.python.compiler.tensorrt import trt_convert as trt
converter = trt.TrtGraphConverter(input_saved_model_dir=input_saved_model_dir)
converter.convert()
converter.save(output_saved_model_dir)
```

Frozen graph:

```
from tensorflow.python.compiler.tensorrt import trt_convert as trt
converter = trt.TrtGraphConverter(
    input_graph_def=frozen_graph,
    nodes_blacklist=['logits', 'classes'])
frozen_graph = converter.convert()
```



Installing TF-TRT

NVIDIA containers of TensorFlow are built with enabling TensorRT, which means TF-TRT is part of the TensorFlow binary in the container and can be used out of the box. The container has all the software dependencies required to run TF-TRT

Activities Firefox Web Browser Wed 13:31

Installation — Tor × How to Use T × Google Slides × My Drive - Go × Accelerating I × Fundamenta × Introducing T × GitHub - tens × Accelerating Info × Build from sou × TensorFlow [× +

https://catalog.ngc.nvidia.com/orgs/nvidia/containers/tensorflow

NVIDIA NGC | CATALOG Select a context alokseetharam

CATALOG Explore Catalog Collections Containers Helm Charts Models Resources

Catalog > Containers > TensorFlow

TensorFlow Pull Tag

Overview Tags Layers Security Scanning Related Collections

TensorFlow Accelerated with NVIDIA

Description
TensorFlow is an open source platform for machine learning. It provides comprehensive tools and libraries in a flexible architecture allowing easy deployment across a variety of platforms and devices.

Publisher
Google Brain Team

Latest Tag
22.03-rt1-py3

Modified
March 29, 2022

Compressed Size
6.42 GB

Multinode Support
Yes

Overview
TensorFlow is an open source platform for machine learning. It provides comprehensive tools and libraries in a flexible architecture allowing easy deployment across a variety of platforms and devices. NGC Containers are the easiest way to get started with TensorFlow. The TensorFlow NGC Container comes with all dependencies included, providing an easy place to start developing common applications, such as conversational AI, natural language processing (NLP), recommenders, and computer vision.

The TensorFlow NGC Container is optimized for GPU acceleration, and contains a validated set of libraries that enable and optimize GPU performance. This container may also contain modifications to the TensorFlow source code in order to maximize performance and compatibility. This container also contains software for accelerating ETL ([DALI](#), [RAPIDS](#)), Training ([cuDNN](#), [NCCL](#)), and Inference ([TensorRT](#)) workloads.

Prerequisites

Using the TensorFlow NGC Container requires the host system to have the following installed:

- [Docker Engine](#)
- [NVIDIA GPU Drivers](#)
- [NVIDIA Container Toolkit](#)

For supported versions, see the [Framework Containers Support Matrix](#) and the [NVIDIA Container Toolkit Documentation](#).

No other installation, compilation, or dependency management is required. *It is not necessary to install the NVIDIA CUDA Toolkit.*

Running TensorFlow



Conversion Parameters in TF-TRT

Parameters that can be passed to `saved_model_cli` and `TrtGraphConverter`.

- `precision_mode`: The precision mode to use (FP32, FP16, or INT8)
- `minimum_segment_size`: The minimum number of TensorFlow nodes required for a TensorRT subgraph to be valid.
- `is_dynamic_op`: TensorRT engines are converted and built at model run time instead of during the `converter.convert()` call. This is required if there are tensors with unknown or dynamic shapes.
- `use_calibration`: Only used if `precision_mode='INT8'`. If True, a calibration graph will be created, and `converter.calibrate()` should be called. This is the recommended option. If False, all tensors that will not be fused must have quantization nodes.
- `max_batch_size`: Used when `is_dynamic_op=False`. This is the maximum batch size for TensorRT engines. At run time, smaller batch sizes can be used, but a larger batch size will result in an error.
- `maximum_cached_engines`: Used when `is_dynamic_op=True`. This limits the number of TensorRT engines that are cached, per `TRTEngineOp`.



Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)(pytorch)





DEMO





Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)



NGC Catalog

Register Now

Getting Started

 TensorFlow

Accelerated with
 NVIDIA

TensorFlow
 Container
TensorFlow is an open source platform for machine learning. It provides comprehensive tools and libraries in a flexible architecture allowing easy...

[View Labels](#) [Pull Tag](#)

 PyTorch

Accelerated with
 NVIDIA

PyTorch
 Container
PyTorch is a GPU accelerated tensor computational framework. Functionality can be extended with common Python libraries such as NumPy and SciPy....

[View Labels](#) [Pull Tag](#)



Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)

Catalog > Containers > PyTorch

PyTorch

Pull



Description

PyTorch is a GPU accelerated tensor computational framework. Functionality can be extended with common Python libraries such as NumPy and SciPy. Automatic differentiation is done with a tape-based system at the functional and neural network layer levels.

Publisher

Facebook

Latest Tag

22.03-py3

Modified

March 20, 2022

Overview

Tags

Layers

Security Scanning

Related Collections

Search tags...



22.03-py3



03/29/2022 5:31 AM

6.59 GB

2 Architectures



22.02-py3



02/25/2022 8:34 AM

6.51 GB

2 Architectures



22.01-py3



01/28/2022 11:05 PM

6.61 GB

2 Architectures



21.12-py3



12/21/2021 8:26 AM

6.49 GB

2 Architectures



Pull Tag



Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)

Pull Docker Container :

```
vinayak@ROG:~$ docker pull nvcr.io/nvidia/pytorch:21.12-py3
21.12-py3: Pulling from nvidia/pytorch
Digest: sha256:a8da7de491196b61e06909c39bcccc0a1c5c4e0a89ecfb2d55a56164bafa9fc9
Status: Image is up to date for nvcr.io/nvidia/pytorch:21.12-py3
nvcr.io/nvidia/pytorch:21.12-py3
```



Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)

```
vinayak@ROG:~$ docker run --net=host --gpus all -it --rm -v /home:/container_dir nvcr.io/nvidia/pytorch:21.12-py3

=====
== PyTorch ==
=====

NVIDIA Release 21.12 (build 29870972)
PyTorch Version 1.11.0a0+b6df043

Container image Copyright (c) 2021, NVIDIA CORPORATION & AFFILIATES. All rights reserved.

Copyright (c) 2014-2021 Facebook Inc.
Copyright (c) 2011-2014 Idiap Research Institute (Ronan Collobert)
Copyright (c) 2012-2014 Deepmind Technologies (Koray Kavukcuoglu)
Copyright (c) 2011-2012 NEC Laboratories America (Koray Kavukcuoglu)
Copyright (c) 2011-2013 NYU (Clement Farabet)
Copyright (c) 2006-2010 NEC Laboratories America (Ronan Collobert, Leon Bottou, Iain Melvin, Jason Weston)
Copyright (c) 2006 Idiap Research Institute (Samy Bengio)
Copyright (c) 2001-2004 Idiap Research Institute (Ronan Collobert, Samy Bengio, Johnny Mariethoz)
Copyright (c) 2015 Google Inc.
Copyright (c) 2015 Yangqing Jia
Copyright (c) 2013-2016 The Caffe contributors
All rights reserved.

NVIDIA Deep Learning Profiler (dlprof) Copyright (c) 2021, NVIDIA CORPORATION & AFFILIATES. All rights reserved.

Various files include modifications (c) NVIDIA CORPORATION & AFFILIATES. All rights reserved.

This container image and its contents are governed by the NVIDIA Deep Learning Container License.
By pulling and using the container, you accept the terms and conditions of this license:
https://developer.nvidia.com/ngc/nvidia-deep-learning-container-license

WARNING: CUDA Minor Version Compatibility mode ENABLED.
Using driver version 470.103.01 which has support for CUDA 11.4. This container
was built with CUDA 11.5 and will be run in Minor Version Compatibility mode.
CUDA Forward Compatibility is preferred over Minor Version Compatibility for use
with this container but was unavailable:
[[Forward compatibility was attempted on non supported HW (CUDA_ERROR_COMPAT_NOT_SUPPORTED_ON_DEVICE) cuInit()=804]]
See https://docs.nvidia.com/deploy/cuda-compatibility/ for details.
```



Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)



Inference with CPU, CUDA and TensorRT

step1: load a pre-trained neural network

```
import torch
from torchvision import models, transforms

# using cpu
#model = models.resnet50(pretrained=True)

# using gpu
model = models.resnet50(pretrained=True).to("cuda")
```



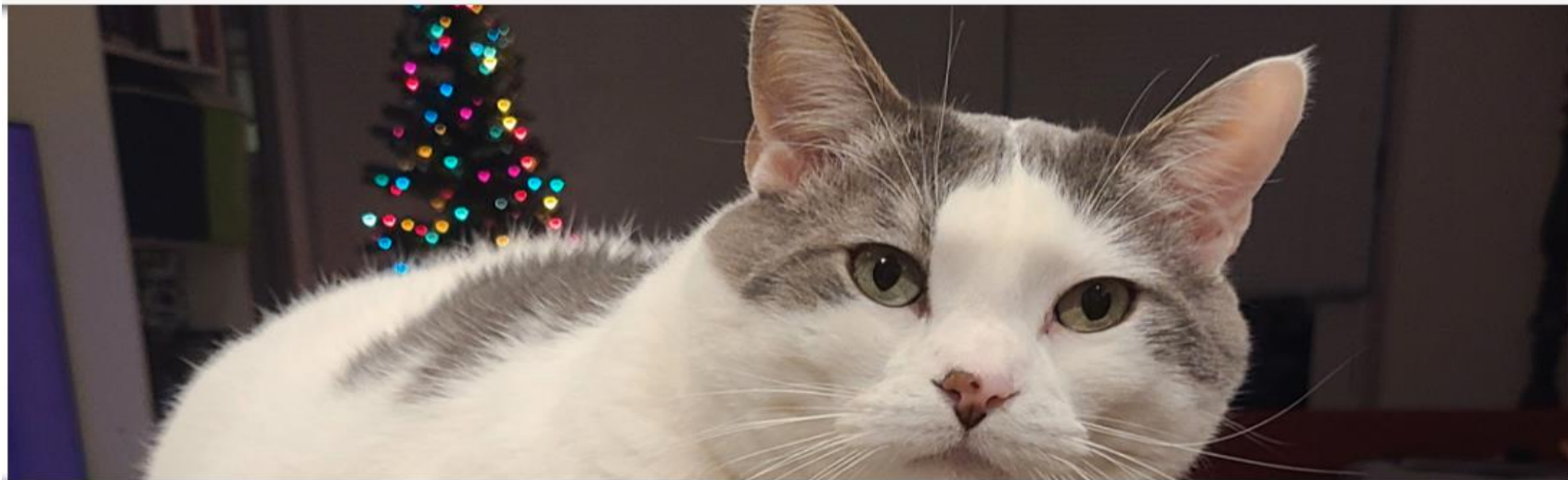


Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)



step 2: load an example image for predition

```
from PIL import Image  
  
img = Image.open("img1.jpg")  
img
```





Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)

step 3: transform example image (pre-processing)

```
transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
])

img = transform(img)
print(img.shape)

torch.Size([3, 224, 224])
```



Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)

step 4: set a batch size to example image

```
# using cpu


```

```
torch.Size([1, 3, 224, 224])
```

step 5: make a prediction and get probabilities for each class

```
model.eval()
with torch.no_grad():
    outputs = model(img_batch)
prob = torch.nn.functional.softmax(outputs[0], dim=0)
```



Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)

```
: import pandas as pd

categories = pd.read_csv('https://raw.githubusercontent.com/pytorch/hub/master/imagenet_classes.txt', header=None)
categories[0][2]

: 'great white shark'
```

step 7: extract top 5 probabilities and map them to the appropriate class name

```
: topk = 5

probs, classes = torch.topk(prob, topk)
for i in range(topk):
    probability = probs[i].item()
    class_label = categories[0][int(classes[i])]
    print("{} {}".format(int(probability*100), class_label))
```

```
%37 Egyptian cat
%19 tabby
%11 tiger cat
%4 Siamese cat
%3 carton
```




Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)

step 8: define a speed test benchmark function & adjust it

source: ResNet50_example.ipynb

or: <https://github.com/NVIDIA/Torch-TensorRT/blob/master/notebooks/Resnet50-example.ipynb>

```
#copy from TRT
import time
import numpy as np

import torch.backends.cudnn as cudnn
cudnn.benchmark = True

def benchmark(model, device="cuda", input_shape=(32, 3, 224, 224), dtype='fp32', nwarmup=50, nruns=100):
    input_data = torch.randn(input_shape)
    input_data = input_data.to(device)

    print("Warm up ...")
    with torch.no_grad():
        for _ in range(nwarmup):
            features = model(input_data)
    torch.cuda.synchronize()
    print("Start timing ...")
    timings = []
    with torch.no_grad():
        for i in range(1, nruns+1):
            start_time = time.time()
            features = model(input_data)
            torch.cuda.synchronize()
            end_time = time.time()
            timings.append(end_time - start_time)
            if i%10==0:
                print('Iteration %d/%d, ave batch time %.2f ms'%(i, nruns, np.mean(timings)*1000))

    print("Input shape:", input_data.size())
    print("Output features size:", features.size())
    print('Average batch time: %.2f ms'%(np.mean(timings)*1000))
```



Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)

step 9: find out CPU model speed benchmarks

```
] : #CPU benchmarks
benchmark(model, device="cpu")

Warm up ...
Start timing ...
Iteration 10/100, ave batch time 640.36 ms
Iteration 20/100, ave batch time 642.35 ms
Iteration 30/100, ave batch time 645.49 ms
Iteration 40/100, ave batch time 646.09 ms
Iteration 50/100, ave batch time 645.95 ms
Iteration 60/100, ave batch time 645.85 ms
Iteration 70/100, ave batch time 646.04 ms
Iteration 80/100, ave batch time 646.32 ms
Iteration 90/100, ave batch time 646.03 ms
Iteration 100/100, ave batch time 645.74 ms
Input shape: torch.Size([32, 3, 224, 224])
Output features size: torch.Size([32, 1000])
Average batch time: 645.74 ms
```



Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)

step 10: find out CUDA model speed benchmarks

```
: #CUDA benchmarks  
model = model.to("cuda")  
benchmark(model)
```

```
Warm up ...  
Start timing ...  
Iteration 10/100, ave batch time 92.37 ms  
Iteration 20/100, ave batch time 92.42 ms  
Iteration 30/100, ave batch time 92.32 ms  
Iteration 40/100, ave batch time 94.92 ms  
Iteration 50/100, ave batch time 97.94 ms  
Iteration 60/100, ave batch time 97.01 ms  
Iteration 70/100, ave batch time 99.83 ms  
Iteration 80/100, ave batch time 101.69 ms  
Iteration 90/100, ave batch time 104.36 ms  
Iteration 100/100, ave batch time 106.73 ms  
Input shape: torch.Size([32, 3, 224, 224])  
Output features size: torch.Size([32, 1000])  
Average batch time: 106.73 ms
```



Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)

step 11: trace and compile CUDA model into a TensorRT model

```
: traced_model = torch.jit.trace(model, [torch.randn((32, 3, 224, 224)).to("cuda")])

: import torch_tensorrt

trt_model = torch_tensorrt.compile(
    traced_model,
    inputs = [torch_tensorrt.Input((32, 3, 224, 224), dtype=torch.float32)],
    enabled_precisions = {torch.float32}
)
```



Benchmarking CPU , GPU (CUDA) , Tensor RT performance : (Demo)

```
benchmark(trt_model)
```

Warm up ...

Start timing ...

Iteration 10/100, ave batch time 76.66 ms

Iteration 20/100, ave batch time 75.95 ms

Iteration 30/100, ave batch time 73.84 ms

Iteration 40/100, ave batch time 72.74 ms

Iteration 50/100, ave batch time 72.09 ms

Iteration 60/100, ave batch time 71.67 ms

Iteration 70/100, ave batch time 71.44 ms

Iteration 80/100, ave batch time 71.20 ms

Iteration 90/100, ave batch time 71.07 ms

Iteration 100/100, ave batch time 71.52 ms

Input shape: torch.Size([32, 3, 224, 224])

Output features size: torch.Size([32, 1000])

Average batch time: 71.52 ms





Demo of Workflow for optimizing Tensorflow model to TensorRT (tensor graph)





Demo of Workflow for optimizing Tensorflow model to TensorRT:-

- Pull The TensorFlow Image :

```
vinayak@ROG:~$ docker pull nvcr.io/nvidia/pytorch:21.12-py3
21.12-py3: Pulling from nvidia/pytorch
Digest: sha256:a8da7de491196b61e06909c39bcccc0a1c5c4e0a89ecfb2d55a56164bafa9fc9
Status: Image is up to date for nvcr.io/nvidia/pytorch:21.12-py3
nvcr.io/nvidia/pytorch:21.12-py3
```



Demo of Workflow for optimizing Tensorflow model to TensorRT:-

- Run The Docker Container of TensorFlow by NGC:

```
vinayak@ROG:~$ docker run --net=host --gpus all -it --rm -v /home/vinayak/Tensorflow-TensorRT:/home/workspace nvcr.io/nvidia/tensorflow:21.12-tf2-py3

=====
== TensorFlow ==
=====

NVIDIA Release 21.12-tf2 (build 29871111)
TensorFlow Version 2.6.2

Container image Copyright (c) 2021, NVIDIA CORPORATION & AFFILIATES. All rights reserved.
Copyright 2017-2021 The TensorFlow Authors. All rights reserved.

NVIDIA Deep Learning Profiler (dlprof) Copyright (c) 2021, NVIDIA CORPORATION & AFFILIATES. All rights reserved.

Various files include modifications (c) NVIDIA CORPORATION & AFFILIATES. All rights reserved.

This container image and its contents are governed by the NVIDIA Deep Learning Container License.
By pulling and using the container, you accept the terms and conditions of this license:
https://developer.nvidia.com/ngc/nvidia-deep-learning-container-license

WARNING: CUDA Minor Version Compatibility mode ENABLED.
Using driver version 470.103.01 which has support for CUDA 11.4. This container
was built with CUDA 11.5 and will be run in Minor Version Compatibility mode.
CUDA Forward Compatibility is preferred over Minor Version Compatibility for use
with this container but was unavailable:
[[Forward compatibility was attempted on non supported HW (CUDA_ERROR_COMPAT_NOT_SUPPORTED_ON_DEVICE) cuInit()=804]]
See https://docs.nvidia.com/deploy/cuda-compatibility/ for details.

NOTE: MOFED driver for multi-node communication was not detected.
Multi-node communication performance may be reduced.

NOTE: The SHMEM allocation limit is set to the default of 64MB. This may be
insufficient for TensorFlow. NVIDIA recommends the use of the following flags:
docker run --gpus all --ipc=host --ulimit memlock=-1 --ulimit stack=67108864 ...
```




Demo of Workflow for optimizing Tensorflow model to TensorRT:-

- Run Jupyter Notebook :

```
root@ROG:/workspace# jupyter notebook --allow-root --ip 0.0.0.0 --port 8888
[I 06:41:29.942 NotebookApp] Writing notebook server cookie secret to /root/.local/share/jupyter/runtime/notebook_cookie_secret
[I 06:41:30.632 NotebookApp] jupyter_tensorboard extension loaded.
[I 06:41:30.901 NotebookApp] JupyterLab extension loaded from /usr/local/lib/python3.8/dist-packages/jupyterlab
[I 06:41:30.901 NotebookApp] JupyterLab application directory is /usr/local/share/jupyter/lab
[I 06:41:30.903 NotebookApp] [JupyterText Server Extension] NotebookApp.contents_manager_class is (a subclass of) jupyter.TextFileContentsManager already - OK
[I 06:41:30.904 NotebookApp] Serving notebooks from local directory: /workspace
[I 06:41:30.904 NotebookApp] Jupyter Notebook 6.4.3 is running at:
[I 06:41:30.904 NotebookApp] http://hostname:8888/?token=03854c4fb934078df585e08ff59cbf5a6414c6b531027a29
[I 06:41:30.904 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 06:41:30.907 NotebookApp]
```

To access the notebook, open this file in a browser:

file:///root/.local/share/jupyter/runtime/nbserver-375-open.html

Or copy and paste this URL:

<http://hostname:8888/?token=03854c4fb934078df585e08ff59cbf5a6414c6b531027a29>



Demo of Workflow for optimizing Tensorflow model to TensorRT:-

Library we use in this Demo

1. Tensorflow 1.12
2. Numpy 1.15.2





Task 1:

(a) Read input: Tensorflow model

(b) Convert to frozen model (*.pb)

```
# import the needed libraries
import tensorflow as tf
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()
import tensorflow.experimental.tensorrt as trt
from tensorflow.python.compiler.tensorrt import trt_convert as trt

from tensorflow.python.platform import gfile

# has to be use this setting to make a session for TensorRT optimization
with tf.Session(config=tf.ConfigProto(gpu_options=tf.GPUOptions(per_process_gpu_memory_fraction=0.50))) as sess:
    # import the meta graph of the tensorflow model
    #saver = tf.train.import_meta_graph("./model/tensorflow/big/model1.meta")
    saver = tf.train.import_meta_graph("./model/tensorflow/small/model_small.meta")
    # then, restore the weights to the meta graph
    #saver.restore(sess, "./model/tensorflow/big/model1")
    saver.restore(sess, "./model/tensorflow/small/model_small")

    # specify which tensor output you want to obtain
    # (correspond to prediction result)
    your_outputs = ["output_tensor/Softmax"]

    # convert to frozen model
    frozen_graph = tf.graph_util.convert_variables_to_constants(
        sess, # session
        tf.get_default_graph().as_graph_def(), # graph+weight from the session
        output_node_names=your_outputs)
    #write the TensorRT model to be used later for inference
    with gfile.GFile("./model/frozen_model.pb", 'wb') as f:
        f.write(frozen_graph.SerializeToString())
    print("Frozen model is successfully stored!")
```



Demo of Workflow for optimizing Tensorflow model to TensorRT:-



Task 2 : Optimize the frozen model to TensorRT graph

```
# convert (optimize) frozen model to TensorRT model
trt_graph = trt.create_inference_graph(
    input_graph_def=frozen_graph,# frozen model
    outputs=your_outputs,
    max_batch_size=2,# specify your max batch size
    max_workspace_size_bytes=2*(10**9),# specify the max workspace
    precision_mode="INT8") # precision, can be "FP32" (32 floating point precision) or "FP16"

#write the TensorRT model to be used later for inference
```





Demo of Workflow for optimizing Tensorflow model to TensorRT:-

- Write Tensor RT Model :

```
with gfile.GFile("./model/TensorRT_model.pb", 'wb') as f:  
    f.write(trt_graph.SerializeToString())  
print("TensorRT model is successfully stored!")
```

TensorRT model is successfully stored!



Demo of Workflow for optimizing Tensorflow model to TensorRT:-

nsisting of 9 nodes by TRTEngineOp_5_1.

2022-03-29 11:43:09.378222: I tensorflow/core/grappler/optimizers/meta_optimizer.cc:1164] Optimization results

r grappler item: tf_graph

constant_folding: Graph size after: 36 nodes (-10), 36 edges (-10), time = 2.508ms.

layout: Graph size after: 41 nodes (5), 41 edges (5), time = 1.946ms.

constant_folding: Graph size after: 41 nodes (0), 41 edges (0), time = 0.977ms.

TensorRTOptimizer: Graph size after: 13 nodes (-28), 13 edges (-28), time = 5030.22607ms.

constant_folding: Graph size after: 13 nodes (0), 13 edges (0), time = 0.857ms.

Optimization results for grappler item: TRTEngineOp_5_0_native_segment

constant_folding: Graph size after: 24 nodes (0), 23 edges (0), time = 0.603ms.

layout: Graph size after: 24 nodes (0), 23 edges (0), time = 0.662ms.

constant_folding: Graph size after: 24 nodes (0), 23 edges (0), time = 0.558ms.

TensorRTOptimizer: Graph size after: 24 nodes (0), 23 edges (0), time = 0.071ms.

constant_folding: Graph size after: 24 nodes (0), 23 edges (0), time = 0.588ms.

Optimization results for grappler item: TRTEngineOp_5_1_native_segment

constant_folding: Graph size after: 11 nodes (0), 10 edges (0), time = 0.943ms.

layout: Graph size after: 11 nodes (0), 10 edges (0), time = 0.592ms.

constant_folding: Graph size after: 11 nodes (0), 10 edges (0), time = 0.602ms.

TensorRTOptimizer: Graph size after: 11 nodes (0), 10 edges (0), time = 0.06ms.

constant_folding: Graph size after: 11 nodes (0), 10 edges (0), time = 0.625ms.



Demo of Workflow for optimizing Tensorflow model to TensorRT:-

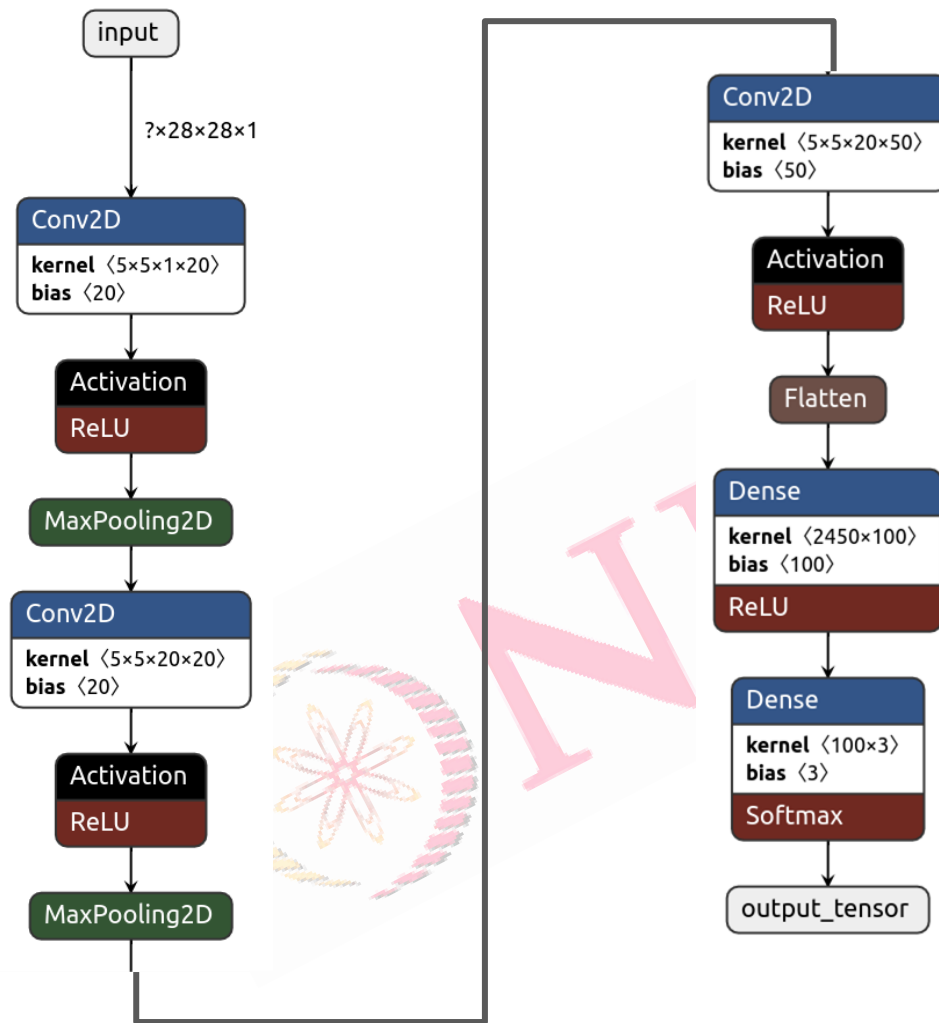


Count how many nodes/operations before and after optimization :

```
# check how many ops of the original frozen model
all_nodes = len([1 for n in frozen_graph.node])
print("numb. of all_nodes in frozen graph:", all_nodes)

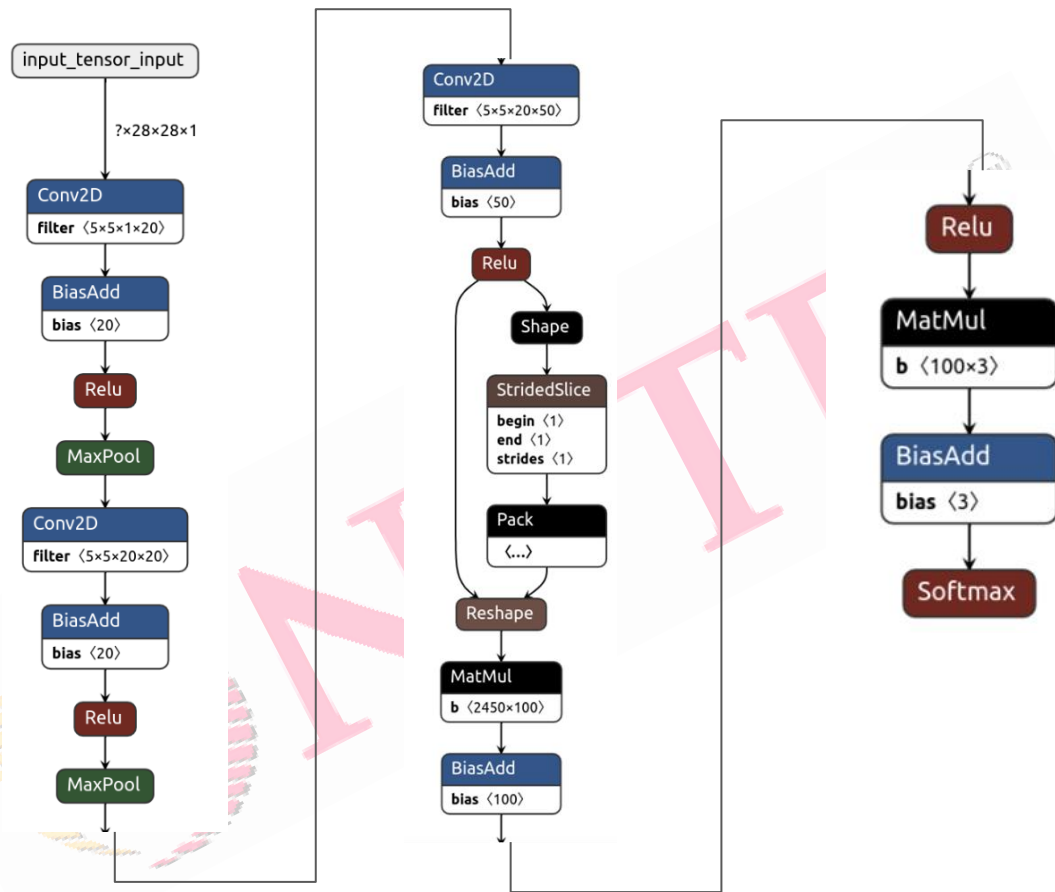
# check how many ops that is converted to TensorRT engine
trt_engine_nodes = len([1 for n in trt_graph.node if str(n.op) == 'TRTEngineOp'])
print("numb. of trt_engine_nodes in TensorRT graph:", trt_engine_nodes)
all_nodes = len([1 for n in trt_graph.node])
print("numb. of all_nodes in TensorRT graph:", all_nodes)
```

```
numb. of all_nodes in frozen graph: 46
numb. of trt_engine_nodes in TensorRT graph: 2
numb. of all_nodes in TensorRT graph: 13
```



Initial Model

Frozen Model :



Converted TensorRT Model :

